

Haskell における ADT と GADT

Hiroki Funashima

Monday, May 4 2026

1 はじめに

Haskell における ADT と GADT は、どちらもデータ型を定義するための仕組みである。しかし、両者の役割は同じではない。

大まかに言えば、ADT は「値の形」を表すための基本的な仕組みであり、GADT はそれを拡張して、「各コンストラクタがどの型を返すか」まで個別に指定できる仕組みである。

基本的な見方

ADT データの構文的な形を表す。

GADT データの構文的な形に加えて、型レベルの意味や制約を表す。

したがって、GADT は ADT の代替というよりも、ADT をより精密にした一般化と考えるとよい。

2 ADT とは何か

ADT は *Algebraic Data Type*，すなわち代数的データ型である。Haskell で通常使う `data` 宣言は、基本的に ADT である。

たとえば、次のような型を考える。

```
data BoolLike
  = MyTrue
  | MyFalse
deriving (Eq, Show)
```

この型の値は、`MyTrue` または `MyFalse` のいずれかである。これは、値の可能な形を列挙している。

このような型は、数学的には直和型に対応する。すなわち、

$$\text{BoolLike} \cong \text{MyTrue} + \text{MyFalse}$$

のように見ることができる。

3 直積型としての ADT

ADT は、直和だけでなく直積も表せる。たとえば、利得行列のセルを表す型を考える。

```
data Cell = Cell
  { cellRowStrategy :: RowStrategy
  , cellColStrategy :: ColStrategy
  , cellPayoff      :: Payoff
  }
deriving (Eq, Show)
```

この Cell は、行プレイヤーの戦略、列プレイヤーの戦略、および利得をひとまとめたデータである。

数学的には、これは概念的に次のような直積型と見なせる。

$$\text{Cell} \cong \text{RowStrategy} \times \text{ColStrategy} \times \text{Payoff}$$

このように、ADT は直和と直積を組み合わせて、対象世界の構造を型として表現する仕組みである。

4 通常の ADT による式の表現

簡単な式を ADT で表してみる。整数リテラル、真偽値リテラル、加算、等号判定を持つ式を考える。

```
data Expr
  = IntLit Int
  | BoolLit Bool
  | Add Expr Expr
  | Equal Expr Expr
deriving (Eq, Show)
```

この定義により、次のような式を表せる。

```
IntLit 3

BoolLit True
```

```
Add (IntLit 1) (IntLit 2)
```

```
Equal (IntLit 1) (IntLit 2)
```

しかし、この定義では、次のような不自然な式も構成できてしまう。

```
Add (BoolLit True) (IntLit 3)
```

これは「真偽値と整数を足す」という意味不明な式である。しかし、型としてはどちらも `Expr` なので、Haskell の型システムはこの時点では拒否できない。

つまり、通常の ADT では、

式の構文的な形は表せるが、その式が型として正しいかまでは十分に表せない。

という限界がある。

5 ADT による評価関数の問題

通常の ADT で式を定義した場合、評価結果の型を一つに決めにくい。

たとえば、次のような評価関数を書きたいとする。

```
eval :: Expr -> ?
```

しかし、式によって評価結果は異なる。整数式を評価すれば `Int` になり、真偽値式を評価すれば `Bool` になる。

そのため、通常は次のような補助的な値型を作ることになる。

```
data Value
  = VInt Int
  | VBool Bool
  deriving (Eq, Show)
```

すると、評価関数は次のように書ける。

```
eval :: Expr -> Value
eval (IntLit n)    = VInt n
eval (BoolLit b)   = VBool b
eval (Add e1 e2) =
  case (eval e1, eval e2) of
    (VInt x, VInt y) -> VInt (x + y)
    _                 -> error "type error"
eval (Equal e1 e2) =
  case (eval e1, eval e2) of
    (VInt x, VInt y) -> VBool (x == y)
    _                 -> error "type error"
```

この実装では、型の不整合を実行時に検査している。つまり、コンパイル時には排除できなかった誤りを、実行時の `error` で処理している。

これは、ADT による単純な構文木表現の限界である。

6 GADT とは何か

GADT は *Generalized Algebraic Data Type*，すなわち一般化代数的データ型である。

GADT を使うには、通常、次の言語拡張を指定する。

```
{-# LANGUAGE GADTs #-}
```

GADT では、コンストラクタごとに型シグネチャを明示する。

先ほどの式を GADT で書くと、次のようになる。

```
data Expr a where
  IntLit  :: Int  -> Expr Int
  BoolLit :: Bool -> Expr Bool
  Add     :: Expr Int -> Expr Int -> Expr Int
  Equal   :: Expr Int -> Expr Int -> Expr Bool
```

ここで `Expr` は型引数 `a` を持つ。この `a` は、「その式を評価すると何型の値が得られるか」を表す。たとえば、

```
IntLit 3
```

の型は、

```
Expr Int
```

である。一方、

```
BoolLit True
```

の型は、

```
Expr Bool
```

である。

つまり、GADT では、式の構文木に「評価結果の型」を埋め込むことができる。

7 GADT による不正な式の排除

GADT で定義した `Expr` では、次の式は正しい。

```
Add (IntLit 1) (IntLit 2)
```

これは `Expr Int` 型である。

一方、次の式は型エラーになる。

```
Add (BoolLit True) (IntLit 3)
```

理由は、`Add` の型が次のように定義されているからである。

```
Add :: Expr Int -> Expr Int -> Expr Int
```

`Add` の左右の引数は、どちらも `Expr Int` でなければならない。しかし、

```
BoolLit True
```

の型は `Expr Bool` である。

したがって、`Add` の引数として使うことはできない。

このように、GADT を用いると、

意味的に不正な構文木を、実行時ではなくコンパイル時に拒否できる。

8 GADT による評価関数

GADT を使うと、評価関数は非常に自然に書ける。

```
eval :: Expr a -> a
eval (IntLit n)    = n
eval (BoolLit b)   = b
eval (Add e1 e2)   = eval e1 + eval e2
eval (Equal e1 e2) = eval e1 == eval e2
```

ここで重要なのは、型が

```
eval :: Expr a -> a
```

となっていることである。

これは、

`Expr a` 型の式を評価すると、`a` 型の値が得られる。

という仕様をそのまま型で表している。

たとえば、

```
eval (IntLit 3)
```

の結果は `Int` であり、

```
eval (BoolLit True)
```

の結果は Bool である。

通常の ADT では Value のような補助型を用意する必要があったが、GADT ではその必要がない。型システムが、式と評価結果の対応を保持しているからである。

9 パターンマッチによる型の精密化

GADT の重要な性質は、パターンマッチによって型情報が精密化されることである。

たとえば、

```
eval :: Expr a -> a
eval (IntLit n) = n
```

という枝を考える。

IntLit の型は、

```
IntLit :: Int -> Expr Int
```

である。したがって、この枝では、コンパイラは

$$a \sim \text{Int}$$

であると知る。

そのため、 $n :: \text{Int}$ を返すことができる。

同様に、

```
eval (BoolLit b) = b
```

という枝では、BoolLit の型が

```
BoolLit :: Bool -> Expr Bool
```

であるため、コンパイラは

$$a \sim \text{Bool}$$

であると知る。

このように、GADT では、パターンマッチが単に値を分解するだけでなく、型情報を具体化する役割も持つ。

10 ADT と GADT の構文上の違い

通常の ADT は、次のように書く。

```
data Expr
  = IntLit Int
  | BoolLit Bool
  | Add Expr Expr
  | Equal Expr Expr
```

一方、GADT は次のように書く。

```
data Expr a where
  IntLit  :: Int -> Expr Int
  BoolLit :: Bool -> Expr Bool
  Add     :: Expr Int -> Expr Int -> Expr Int
  Equal   :: Expr Int -> Expr Int -> Expr Bool
```

見た目の違いは、where を使い、各コンストラクタに型シグネチャを書く点である。

しかし、本質的な違いは構文ではない。

本質的な違い

ADT では、各コンストラクタは同じ型を作る。

GADT では、各コンストラクタが返す型を個別に指定できる。

11 通常の ADT は GADT の特殊な場合である

通常の ADT は、GADT の特殊な場合と見なせる。

たとえば、通常の Maybe 型は次のように書ける。

```
data Maybe a
  = Nothing
  | Just a
```

これを GADT 風に書くと、次のようになる。

```
data Maybe a where
  Nothing :: Maybe a
  Just    :: a -> Maybe a
```

この定義では、どちらのコンストラクタも Maybe a を返している。したがって、通常の ADT と本質的に同じである。

一方、GADT では次のように、コンストラクタごとに異なる型引数を返せる。

```
data Expr a where
  IntLit  :: Int -> Expr Int
  BoolLit :: Bool -> Expr Bool
```

この点が、GADT の「一般化」された部分である。

12 GADT が有効な場面

GADT が特に有効なのは、データ構造に型レベルの意味や不変条件を持たせたい場合である。

12.1 型付き抽象構文木

典型例は、型付き抽象構文木である。

```
data Expr a where
  IntLit  :: Int -> Expr Int
  BoolLit :: Bool -> Expr Bool
  If      :: Expr Bool -> Expr a -> Expr a -> Expr a
```

この If では、条件部は必ず Expr Bool でなければならない。また、then 節と else 節は同じ型 Expr a でなければならない。

したがって、

```
If (BoolLit True) (IntLit 1) (IntLit 2)
```

は正しいが、

```
If (IntLit 0) (IntLit 1) (IntLit 2)
```

は型エラーになる。

12.2 状態遷移の制約

GADT は、状態遷移の制約を表す場合にも使える。たとえば、ある処理が

未認証 → 認証済み → 実行可能

という状態を持つとする。

このとき、型レベルで状態を区別することで、「認証済みでなければ実行できない」といった制約を表せる。

12.3 不変条件の表現

GADT は、次のような不変条件を型に反映させたい場合にも有効である。

- 型検査済みの式である。
- 空でないリストである。

- 検証済みの入力である。
- ある状態でのみ実行可能な操作である。
- ある構造的条件を満たすデータである。

ただし、長さ付きベクトルのような高度な例では、GADT に加えて、DataKinds や型レベル自然数などを用いることが多い。

13 ADT と GADT の比較

観点	ADT	GADT
主な役割	値の形を表す	値の形に加えて、型レベルの意味を表す
コンストラクタ	引数の形を列挙する	コンストラクタごとに型シグネチャを書く
返す型	基本的に同じ型を返す	コンストラクタごとに異なる型引数を返せる
パターンマッチ	値の形を分解する	値の形を分解し、型情報も精密化する
典型例	Maybe, Either, リスト, Cell, Game	型付き式, DSL, 状態遷移, 型安全なインタプリタ
利点	単純で分かりやすい	不正な構造をコンパイル時に排除できる
注意点	意味的制約は別途検査が必要	型推論やエラーメッセージが難しくなりやすい

14 使い分けの目安

基本的には、まず ADT で設計するのがよい。

たとえば、ゲーム全体を表す型は、通常の ADT で自然に書ける。

```
data Game = Game
{ rowStrategies :: Vector RowStrategy
, colStrategies :: Vector ColStrategy
, cells          :: Vector (Vector Cell)
}
deriving (Eq, Show)
```

この型は、ゲームがどのような部品から構成されるかを表している。このような構造表現には、通常の ADT が適している。

一方、次のような性質を型で保証したくなった場合には、GADT を検討する。

- この式は必ず Int を返す。
- この式は必ず Bool を返す。

- この操作は特定の状態でしか呼び出せない。
- この構文木は型検査済みである。
- このデータはある不変条件を満たしている。

つまり、ADT は対象の構造を表す道具であり、GADT は対象の構造に加えて、型レベルの制約を表す道具である。

15 まとめ

ADT と GADT の違いは、次のように整理できる。

まとめ

ADT データの構文的な構造を表す。

GADT データの構文的な構造に加えて、型レベルの意味や制約を表す。

ADT では、値がどのような形を取りうるかを列挙する。これは、ゲームのセル、戦略、利得、木構造、式の構文などを表すのに適している。

一方、GADT では、コンストラクタごとに返す型を精密に指定できる。そのため、型付き式、DSL、状態遷移、型安全なインタプリタなどを表すときに有効である。

概念的には、次のように言える。

ADT = データの形を表す仕組み

GADT = データの形と型レベルの意味を同時に表す仕組み

したがって、Haskell における ADT/GADT は、単なるデータ定義の構文ではなく、対象世界の構造、意味、不変条件をどこまで型に反映させるかという設計上の選択に関わる概念である。